



Relative NN-Descent: A Fast Index Construction for Graph-Based Approximate Nearest Neighbor Search

Naoki Ono
n_ono@hal.t.u-tokyo.ac.jp
The University of Tokyo
Tokyo, Japan

Yusuke Matsui
matsui@hal.t.u-tokyo.ac.jp
The University of Tokyo
Tokyo, Japan

ABSTRACT

Approximate Nearest Neighbor Search (ANNS) is the task of finding the database vector that is closest to a given query vector. Graph-based ANNS is the family of methods with the best balance of accuracy and speed for million-scale datasets. However, graph-based methods have the disadvantage of long index construction time. Recently, many researchers have improved the tradeoff between accuracy and speed during a search. However, there is little research on accelerating index construction. We propose a fast graph construction algorithm, Relative NN-Descent (RNN-Descent). RNN-Descent combines NN-Descent, an algorithm for constructing approximate K-nearest neighbor graphs (K-NN graphs), and RNG Strategy, an algorithm for selecting edges effective for search. This algorithm allows the direct construction of graph-based indexes without ANNS. Experimental results demonstrated that the proposed method had the fastest index construction speed, while its search performance is comparable to existing state-of-the-art methods such as NSG. For example, in experiments on the GIST1M dataset, the construction of the proposed method is 2x faster than NSG. Additionally, it was even faster than the construction speed of NN-Descent.

CCS CONCEPTS

• **Information systems** → **Nearest-neighbor search**; • **Computing methodologies** → **Visual content-based indexing and retrieval**.

KEYWORDS

approximate nearest neighbor search, graph-based index

ACM Reference Format:

Naoki Ono and Yusuke Matsui. 2023. Relative NN-Descent: A Fast Index Construction for Graph-Based Approximate Nearest Neighbor Search. In *Proceedings of the 31st ACM International Conference on Multimedia (MM '23)*, October 29–November 3, 2023, Ottawa, ON, Canada. ACM, New York, NY, USA, 9 pages. <https://doi.org/10.1145/3581783.3612290>

1 INTRODUCTION

Approximate Nearest Neighbor Search (ANNS) is the task of finding the database vector that is closest to a given query vector. It

has many applications, from image and text retrieval to machine learning, such as the k-nearest neighbor classification. ANNS has a tradeoff between speed, accuracy, and memory consumption. Typical methods for solving ANNS include hashing-based [1, 10], quantization-based [9, 15], and tree-based [21, 24] approaches.

In recent years, many studies have proposed graph-based ANNS methods. Graph-based methods construct a graph with database vectors as vertices. The edges of the graph connect vectors that are close to each other. The search algorithm finds the approximate nearest neighbor by traversing the graph toward the query. Graph-based ANNS is a family of methods offering the best accuracy and speed tradeoff for million-scale datasets [17, 20].

The weakness of graph-based methods is their long index construction time. For example, consider index construction on the GIST1M [15] dataset, which contains 1 million vectors of 960 dimensions. Construction of HNSW [19], a representative of graph-based indexes, takes approximately 20 minutes on an AWS c6i.4xlarge instance (16 vCPUs, 32GB memory). Speeding up the index construction time is important for many cases. One of those cases is when data is updated frequently. Only a few studies [22] have addressed the issue of efficiently deleting data from a graph-based index. Therefore, when deleting a large amount of data from an index, users must reconstruct the graph from scratch. Fast construction also improves efficiency in finding optimal construction parameters for various datasets. Despite their importance, there is little research on speeding up index construction.

We aim to accelerate the construction of graph indexes. We classify conventional graph-based indexes into two types and show why each is slow to construct. The first type is (1) the refinement-based approach [7, 8, 12, 13, 17]. These methods first construct an initial graph (e.g., an approximate K-NN graph by NN-Descent [5]). Then they refine the K-NN graph using algorithms such as RNG Strategy [8, 11, 19] to obtain the final graph. The refinement-based approach can construct high-performance graph indexes quickly by using K-NN graphs to narrow candidate edges of the index. However, the construction of the initial K-NN graph takes time. The second type is (2) the direct approach [14, 18, 19], which constructs a graph index directly. HNSW [19] is a typical method of this type. HNSW sequentially adds new vectors to the index. Here, HNSW finds neighbors of the new data by performing ANNS on the index under construction. HNSW is one of the best-performing graph-based methods. However, constructing a high-performance graph requires accurate ANNS, which is time-consuming because of the tradeoff between ANNS speed and accuracy.

We propose a new graph index construction algorithm, Relative NN-Descent (RNN-Descent). RNN-Descent simultaneously solves

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

MM '23, October 29–November 3, 2023, Ottawa, ON, Canada

© 2023 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 979-8-4007-0108-5/23/10...\$15.00

<https://doi.org/10.1145/3581783.3612290>

the problems of the refinement-based approach and the direct approach. First, RNN-Descent directly constructs the graph-based index to avoid the construction of K-NN graphs, which is a problem of the refinement-based approach. Second, RNN-Descent constructs the graph using the algorithm derived from NN-Descent. This algorithm does not use ANNS, unlike the direct approach. The technical idea of the proposed method is to execute the construction and improvement of K-NN graphs in the refinement-based approach simultaneously. Specifically, RNN-Descent combines NN-Descent, an algorithm for constructing K-NN graphs, and RNG Strategy, an algorithm for selecting edges valid for ANNS. This idea allows the algorithm to directly construct the index without using ANNS. The neighborhood update algorithm also naturally preserves connectivity, which is important for graph index performance.

Our experiments showed that the proposed method has the equivalent search performance to the conventional methods, whereas its construction is much faster than theirs. For example, on the GIST1M dataset, the construction speed of the proposed method was about twice that of NSG [8], one of the state-of-the-art methods. Remarkably, the construction speed of the proposed method was even faster than that of the K-NN graph. We also constructed graphs on the SIFT20M [16] dataset to analyze their performance on large datasets.

Our contributions are as follows:

- We tackle the fast construction of graph indexes. Despite its importance, previous researches have yet to address speeding up the construction of graphs.
- We propose RNN-Descent, a novel graph index construction algorithm. RNN-Descent simultaneously solves the problems of the conventional graph-based approaches: (1) the refinement-based approach and (2) the direct approach.
- Our experiments demonstrate that RNN-Descent is the fastest construction algorithm, and the constructed index has a compatible search performance with conventional methods.

2 RELATED WORK

2.1 Approximate Nearest Neighbor Search (ANNS)

We first formulate the nearest neighbor search (NNS) problem. Let $q \in \mathbb{R}^d$ as query vector, $n \in \mathbb{Z}$ be the number of database vectors, $X = \{x_1, \dots, x_n\} \subset \mathbb{R}^d$ be the database vectors and $\text{dist} : \mathbb{R}^d \times \mathbb{R}^d \rightarrow \mathbb{R}$ be a distance function. Here, NNS is a task that returns the ID of the database vector $x_{i^*} \in X$ closest to q :

$$i^* = \underset{i \in \{1, \dots, n\}}{\operatorname{argmin}} \text{dist}(q, x_i) \quad (1)$$

A naive approach to solving NNS is to compute the distance between all the data and the query. However, the time complexity of the exhaustive search is $O(nd)$, which is slow for large and high-dimensional data.

Numerous studies have addressed approximate NNS (ANNS) as a method for large-scale data. ANNS is a method that improves speed significantly at the cost of a slight loss of accuracy. ANNS methods generally have a trade-off between accuracy, speed, and memory consumption. Therefore the appropriate method depends on the

Algorithm 1 SEARCH(G, q, L)

Require: graph $G = (V, E)$, query $q \in \mathbb{R}^d, L \in \mathbb{Z}$

Ensure: approximate nearest neighbor $v^* \in V$

```

1:  $C \leftarrow \text{INITIALIZECANDIDATES}()$ 
2: while True do
3:    $u \leftarrow$  nearest unvisited point to  $q$  in  $C$ 
4:    $U \leftarrow \{v \mid (u, v) \in E\}$ 
5:   for  $v \in U$  do
6:     if  $v$  is not visited then
7:        $C \leftarrow C \cup \{v\}$ 
8:   if  $|C| > L$  then
9:      $C \leftarrow$  top  $L$  nearest points to  $q$  in  $C$ 
10:  if  $C$  is not updated then
11:    break
12: return nearest point to  $q$  in  $C$ 

```

scale of the problem. ANNS methods include hash-based [1, 10], quantization-based [9, 15], and tree-based methods [21, 24].

2.2 Graph-Based ANNS

This section describes graph-based ANNS. Graph-based methods have the best trade-off between accuracy and speed for million-scale datasets [2, 17, 25]. Before searching, graph-based methods construct a graph in which vertices represent database vectors. The edges of the graph connect vectors close to each other. The search algorithm finds the approximate nearest neighbor by traversing the constructed graph toward the query.

Algorithm 1 shows the typical algorithm of graph traversal. Here, $V = \{1, \dots, n\}$ is the graph vertex set, $E \subset V \times V$ is the edge set, and $L \in \mathbb{Z}$ is a hyperparameter. Each $v \in V$ corresponds to a database vector $x_v \in \mathbb{R}^d$. First, the algorithm initializes the set of candidate neighborhood points $C \subset V$ (L1). Each step in the while loop takes the unvisited point closest to the query u from C (L3), then adds unvisited vertices of u 's neighbors to C (L4-7). Subsequently, the algorithm selects the top L closest points to q in C so that the size of C does not exceed L (L8-9). When C is no longer updated, the algorithm terminates the while loop and outputs the nearest neighbor from C (L10-11).

One of the most straightforward graph indexes is the K-nearest neighbor graph (K-NN graph). A K-NN graph has edges from each vertex to the top K vertices closest to it. Since constructing an exact K-NN graph is time-consuming, several methods construct approximate K-NN graphs. NN-Descent [5] is one of the fastest and most accurate methods. NN-Descent improves K-NN graphs step-by-step by dealing with the neighbors of neighbors as candidates for new neighbors. Other approaches include divide-and-conquer [4, 23], hashing-based [6], and sequentially adding vertices by solving ANNS on a subset of the data [26].

Many methods develop different graph structures to improve search performance. The first approach is to improve the approximate K-NN graph. Because simple K-NN graphs perform poorly as ANNS indexes, some methods refine K-NN graphs to a high-performance graph index. NSG [8] extracts the edge candidates from the K-NN graph, then selects the necessary ones using RNG Strategy. RNG Strategy is a widely used heuristic for selecting edges

Algorithm 2 NNDESCENTJOIN(G)**Require:** graph $G = (V, E)$ **Ensure:** edge candidates $E' \subset V \times V$

```

1:  $E' \leftarrow E$ 
2: for  $u \in V$  do
3:    $U \leftarrow \{v \mid (u, v) \in E\}$ 
4:   for all  $(v_1, v_2) \in U \times U, v_1 < v_2$  do
5:     if at least one flag of  $v_1$  or  $v_2$  is “new” then
6:        $E' \leftarrow E' \cup \{(v_1, v_2), (v_2, v_1)\}$ 
7:   set the flag “old” for all vertices in  $U$ 
8: return  $E'$ 

```

Algorithm 3 RNGSTRATEGY(u, U)**Require:** vertex $u \in V$, neighbor candidates $U \subset V$ **Ensure:** selected neighbors $U' \subset U$

```

1: sort  $v \in U$  in ascending order of  $\delta(u, v)$ 
2:  $U' \leftarrow \emptyset$ 
3: for  $v \in U$  do
4:    $f \leftarrow \text{true}$ 
5:   for  $w \in U'$  do
6:     if  $\delta(u, v) \geq \delta(v, w)$  then
7:        $f \leftarrow \text{false}$ 
8:       break
9:   if  $f$  then
10:     $U' \leftarrow U' \cup \{v\}$ 
11: return  $U'$ 

```

based on the distance between neighbors. Related methods include those that focus on the angles between edges [7, 17] or adjust the degree of selection by additional parameters [14]. The second approach is to construct the graph-based index directly. HNSW [19] sequentially adds new data to the index. The construction algorithm finds neighbor candidates of the new vertex by solving ANNS for the current graph index. HNSW adds data sequentially. These methods consider the newly added data as a query vector and solve ANNS for the current graph index to find candidate neighbors for the new data. While these construction methods are simpler than those via K-NN graphs, it is slower for high-dimensional datasets where ANNS is difficult.

3 PRELIMINARY

3.1 NN-Descent

NN-Descent [5] is a fast algorithm for constructing approximate K-NN graphs. The basic idea of NN-Descent is that neighbors of neighbors are likely to be neighbors again. NN-Descent first initializes the graph randomly. It then repeats the join step to find new neighbor candidates and the update step to select the neighbors from the candidates.

Figure 1(a) shows an example of the join operation. For speed, the join operation examines all pairs around an arbitrary vertex instead of checking the neighbors of the neighbors. The vertices v_1, v_2 , and v_3 in Figure 1(a) are neighbors of a neighbor via u . Therefore, the join algorithm adds a bi-directional edge between each vertex pair (v_i, v_j) ($1 \leq i < j \leq 3$).

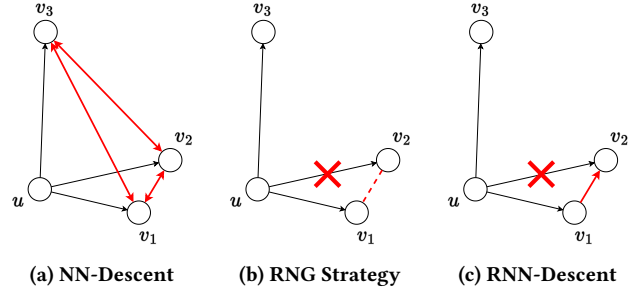


Figure 1: Comparison of methods for constructing graph indices. Vertices v_1, v_2 , and v_3 in the figure are neighbors of vertex u . (a) NN-Descent is a method for constructing approximate K-NN graphs step-by-step by collecting neighbors of neighbors as candidates for new neighbors. For example, vertices v_1 and v_2 are neighbors of a neighbor via u . NN-Descent adds a new bi-directional edge between v_1 and v_2 . (b) RNG Strategy is one of the methods to select edges essential for the search. In the figure, RNG Strategy removes the edge to v_2 because v_1 and v_2 are close enough. (c) RNN-Descent, the proposed method, combines the features of NN-Descent and RNG Strategy. In the figure, it removes the edge (u, v_2) by RNG Strategy and adds a new edge (v_1, v_2) instead.

Algorithm 2 is a pseudo-code for the NN-Descent join operation. The algorithm takes graph $G = (V, E)$ and returns the candidates of new edges. Each loop after Line 2 bidirectionally adds edges for each pair of u 's neighbors. NN-Descent assigns a flag to each neighbor to determine if it is newly added in the most recent step. Each join step adds an edge between neighbor pairs only if either of the flags is “new” (L5). This method allows the join step to check each pair of neighbors only once, speeding up the join step. Since the proposed method incorporates the flags, we describe the algorithm in detail again in Section 4.2.

3.2 RNG Strategy

The RNG Strategy [8, 11, 19] is a method for selecting edges of a graph valid for ANNS. Figure 1(b) shows an image of the RNG Strategy. RNG Strategy reduces edges so that any two neighbors v and w of each vertex u satisfy the following inequalities:

$$\delta(u, v) < \delta(v, w) \wedge \delta(u, w) < \delta(v, w) \quad (2)$$

Here, $\delta(u, v)$ is the distance between u and v :

$$\delta(u, v) = \text{dist}(x_u, x_v) \quad (3)$$

For example, vertices v_1 and v_2 in Figure 1(b) do not satisfy $\delta(u, v_2) < \delta(v_2, v_1)$, so the algorithm deletes the edge from u to v_2 . Intuitively, when v and w are too close to each other, it is likely that an edge already exists between v and w . Then, if there is an edge from u to either v or w , the other is also reachable from u .

Algorithm 3 is a pseudocode for RNG Strategy. The input is neighbor candidates U of vertex u , and the output is selected neighbors U' . First, the algorithm sorts U in ascending order of distance to u (L1). Then, for each vertex v in U , the algorithm determines

Algorithm 4 UPDATENEIGHBORS(G)

Require: graph $G = (V, E)$, vertex $u \in V$

```

1: for  $u \in V$  do
2:    $U \leftarrow \{v \mid (u, v) \in E\}$ 
3:   sort  $v \in U$  in ascending order of  $\delta(u, v)$ 
4:    $U' \leftarrow \emptyset$ 
5:   for  $v \in U$  do
6:      $f \leftarrow \text{true}$ 
7:     for  $w \in U'$  do
8:       if both flags of  $v$  and  $w$  are “old” then
9:         continue
10:      if  $\delta(u, v) \geq \delta(v, w)$  then
11:         $f \leftarrow \text{false}$ 
12:         $E \leftarrow (E \setminus (u, v)) \cup (w, v)$ 
13:        break
14:    if  $f$  then
15:       $U' \leftarrow U' \cup \{v\}$ 
16:  set the flag “old” for all vertices in  $U'$ 

```

whether output candidates set U' includes v (L3-8). Specifically, for each vertex w already added to U' , it checks whether v satisfies the constraint $\delta(u, v) < \delta(v, w)$. If v passes the check, the algorithm adds v to U' (L9-10).

4 METHOD

4.1 Motivation

We categorize conventional graph-based ANNS methods into two main approaches: (1) a refinement-based approach and (2) a direct approach. The refinement-based approach first constructs an approximate K-NN graph and then refines it to obtain a final graph-based index. However, constructing the K-NN graph is time-consuming and increases the overall index construction time. On the other hand, the direct approaches construct the graph-based index without going through the K-NN graph by solving ANNS on the index under construction. However, this approach is also slow because the accuracy of the ANNS must be high to construct a graph-based index with good performance.

We propose a new graph construction algorithm, Relative NN-Descent (RNN-Descent), to solve the above problems. RNN-Descent is faster than the refinement-based approach because it does not go through the K-NN graph. In addition, the proposed method can construct a high-performance graph-based index in less time than the direct approach because it constructs the index without ANNS.

The technical core of the proposed method is the combination of two algorithms mentioned in Section 3: NN-Descent and RNG Strategy. Akin to NN-Descent, RNN-Descent constructs a graph-based index by incrementally improving a randomly initialized graph. However, the update algorithm of RNN-Descent simultaneously performs an edge-adding operation derived from the NN-Descent and an edge-removing operation based on the RNG Strategy. This approach allows the proposed method to directly construct the graph without finding neighbor candidates with ANNS. In addition, the proposed update algorithm naturally guarantees graph connectivity, which is essential for search performance.

Algorithm 5 ADDREVERSEEDGES(G, R)

Require: graph $G = (V, E)$, $R \in \mathbb{Z}$

```

1:  $E \leftarrow E \cup \{(v, u) \mid (u, v) \in E\}$ 
2: set flags of new neighbors to “new”
3: for  $v \in V$  do
4:    $E_u \leftarrow \{(v, u) \mid (v, u) \in E\}$ 
5:   remove top- $R$  shortest edges from  $E_u$ 
6:    $E \leftarrow E \setminus E_u$ 
7: for  $v \in V$  do
8:    $E_u \leftarrow \{(u, v) \mid (u, v) \in E\}$ 
9:   remove top- $R$  shortest edges from  $E_u$ 
10:   $E \leftarrow E \setminus E_u$ 

```

Algorithm 6 RNN-DESCENT(S, R, T_1, T_2)

Require: $S, R, T_1, T_2 \in \mathbb{Z}$

Ensure: graph $G = (V, E)$

```

1:  $G \leftarrow \text{RANDOMGRAPH}(S)$ 
2: initialize all flags to “new”
3: for  $t_1 = 1, \dots, T_1$  do
4:   for  $t_2 = 1, \dots, T_2$  do
5:     UPDATENEIGHBORS( $G$ )
6:   if  $t_1 \neq T_1$  then
7:     ADDREVERSEEDGES( $G, R$ )
8: return  $G$ 

```

Section 4.2 describes the details of the proposed neighbors updating algorithm, and Section 4.3 discusses the reverse edges addition algorithm to avoid suboptimal graphs. Finally, Section 4.4 describes the overall construct and search algorithm.

4.2 Updating neighbors

This section describes the algorithm for updating the neighbors. The idea of the proposed method is to simultaneously perform the neighborhood update algorithm of NN-Descent and the edge selection algorithm of the RNG Strategy.

Figure 1 shows how the proposed algorithm works. We consider neighbors of a vertex u . Normal NN-Descent adds an edge between any two neighborhoods. For example, in Figure 1(a) NN-Descent adds bidirectional edges between v_1, v_2 , and v_3 . However, adding edges between every neighbor pair is useless in constructing a graph-based index. It is because some edges will be eliminated by algorithms such as RNG-Strategy, as the conventional refinement-based approach does. Therefore, the proposed method uses the RNG Strategy concept to add only the necessary edges. According to RNG Strategy, the edge (u, v_2) in Figure 1(b) is not necessary because of the inequality $\delta(u, v_2) > \delta(v_2, v_1)$. The proposed method removes redundant edges and adds proper edges simultaneously. In Figure 1(c), the algorithm removes the edge (u, v_2) and then inserts the edge (v_2, v_1) instead. That is, the proposed method combines NN-Descent and RNG Strategy. Our neighborhood update algorithm also keeps the graph’s connectivity, which is important to improve the performance of ANNS. For instance, in Figure 1(c), v_2 is reachable from u before and after the update algorithm.

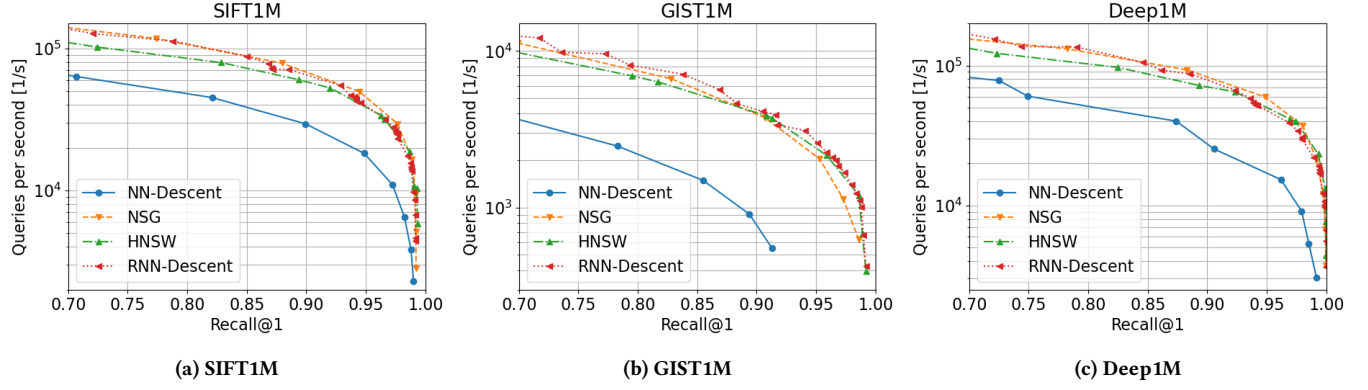


Figure 2: Search performance for graph-based ANNS methods.

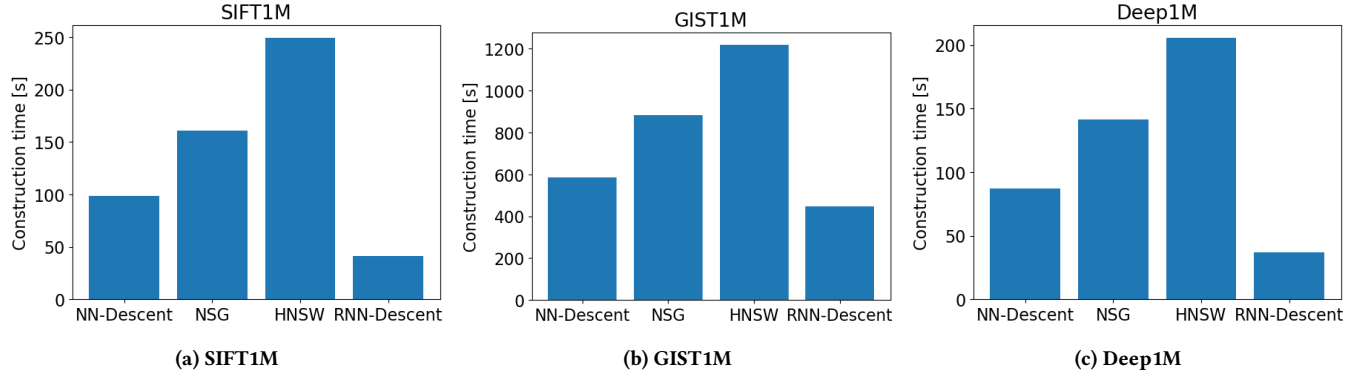


Figure 3: Construction time for graph-based ANNS methods.

Algorithm 4 is a pseudo code for a neighborhood update algorithm. The algorithm takes graph $G = (V, E)$ as input and updates the edge set $E \subset V \times V$. Most of the algorithm is the same as the RNG Strategy. One of the differences is in Line 11. If u 's neighbor v does not satisfy the inequality for some selected vertex w , the algorithm removes edge (u, v) and inserts the edge (v, w) instead. The algorithm adds no edges if the edge (w, v) already exists. However, w is still reachable from u in this case.

Another difference is the introduction of a flag to determine if each neighbor is newly added in the last iteration. This technique derives from the original NN-Descent. If both flags of vertices v and w are “old,” the algorithm skips to calculate the distance between them (L5-6). It is because if v and w are old neighbors, the algorithm has already checked whether v and w satisfy the RNG inequality. At the end of the iteration, the algorithm sets the flag of all neighbors in U' to “old” (L15).

4.3 Adding reverse edges

The problem with the update algorithm in Section 4.2 is that the constructed graph will likely fall into a local optimum with low performance. Here, a local optimum means that all neighbors satisfy the conditions of the RNG Strategy, and the update algorithm will not update edges anymore. Suboptimal graphs have long average edge distances, leading to poor search performance.

Our solution is to add reverse edges to the suboptimal graph. Adding a new edge that does not satisfy Eq 2 allows the update algorithm to restart. Also, since the graph's reverse edges are likely shorter than randomly chosen edges, they are suitable for converging the graph to a better solution.

Algorithm 5 is a pseudo-code for adding the reverse edges. Here, the input $R \in \mathbb{Z}$ is a parameter that controls the number of reduced edges. First, the algorithm adds the reverse edges to the current edge set E (L1). Then, it removes some long edges from E to prevent the number of edges from increasing too much. Specifically, the algorithm reduces the edges so that the in-degree (L3-5) and out-degree are less than or equal to R (L6-8).

4.4 Overall algorithm

Algorithm 6 is the pseudo-code for the entire RNN-Descent algorithm. First, the algorithm initializes the graph randomly (L1). Here, $S \in \mathbb{Z}$ is the out-degree of the initial graph. Each step in the subsequent loop improves the initialized graph G step-by-step. The algorithm in Section 4.2 updates the edge set T_2 times (L4-5). Then, the algorithm in Section 4.3 adds reverse edges to prevent the graph from falling suboptimal (L6-7). After repeating this sequence of steps T_1 times, the algorithm outputs the final graph (L8).

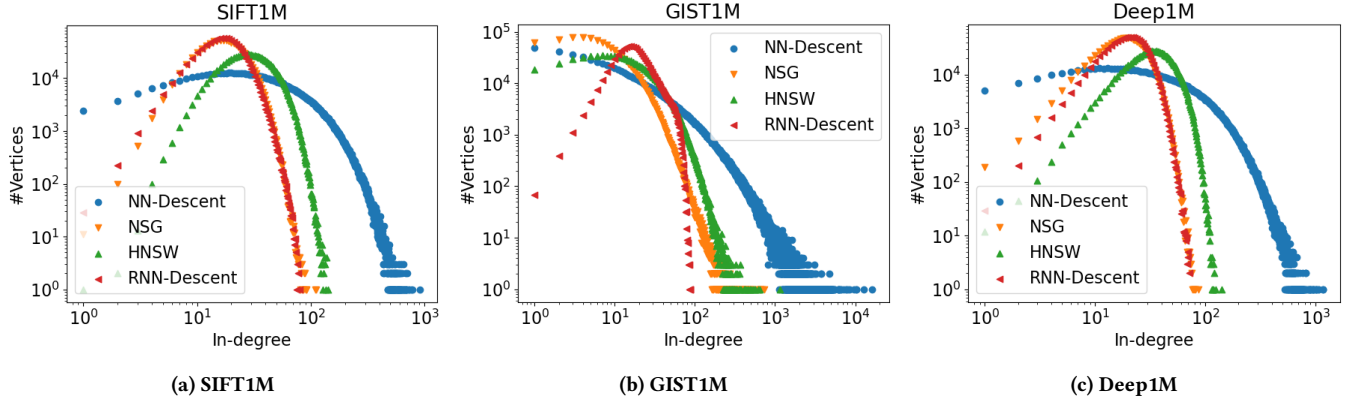


Figure 4: In-degree distribution.

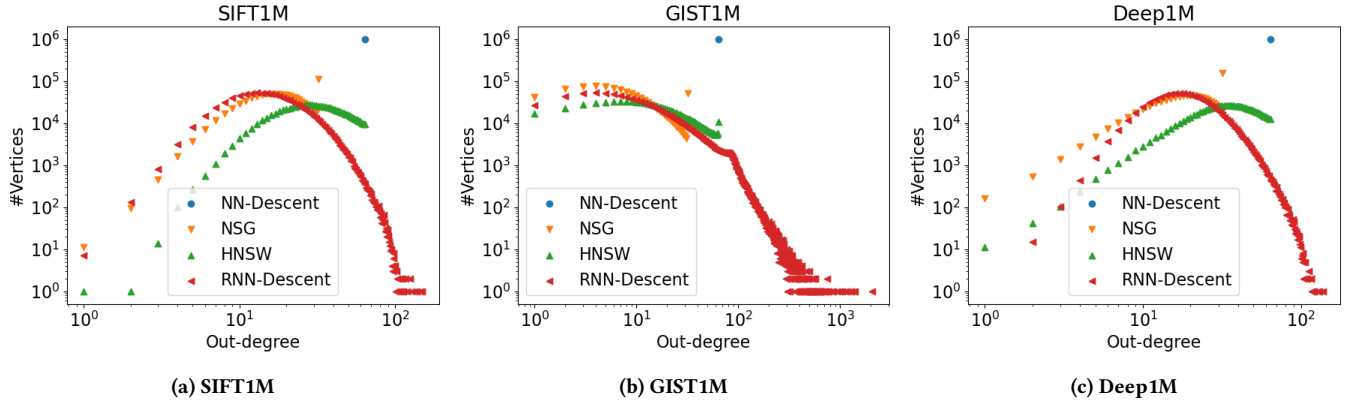


Figure 5: Out-degree distribution.

Table 1: The properties of datasets.

Dataset	Dimensions	#Bases	#Queries
SIFT1M [15]	128	1,000,000	10,000
GIST1M [15]	960	1,000,000	1,000
Deep1M [3]	96	1,000,000	10,000
SIFT20M [16]	128	20,000,000	10,000

Finally, we describe the search algorithm for the constructed indexes. RNN-Descent does not limit the out-degree of the constructed graph. Instead, the search algorithm limits the number of degrees. We replace L4 in Algorithm 1 as follows:

$$U \leftarrow \text{top } K \text{ nearest points to } u \text{ in } \{v | (u, v) \in E\} \quad (4)$$

Here, $K \in \mathbb{Z}$ is a parameter that determines the maximum out-degree. This algorithm allows users to change the maximum out-degree without reconstructing the graph. The optimal degree depends on the dataset, but it is difficult to know this before the construction. In contrast, the proposed method can dynamically determine the optimal out-degree during the search.

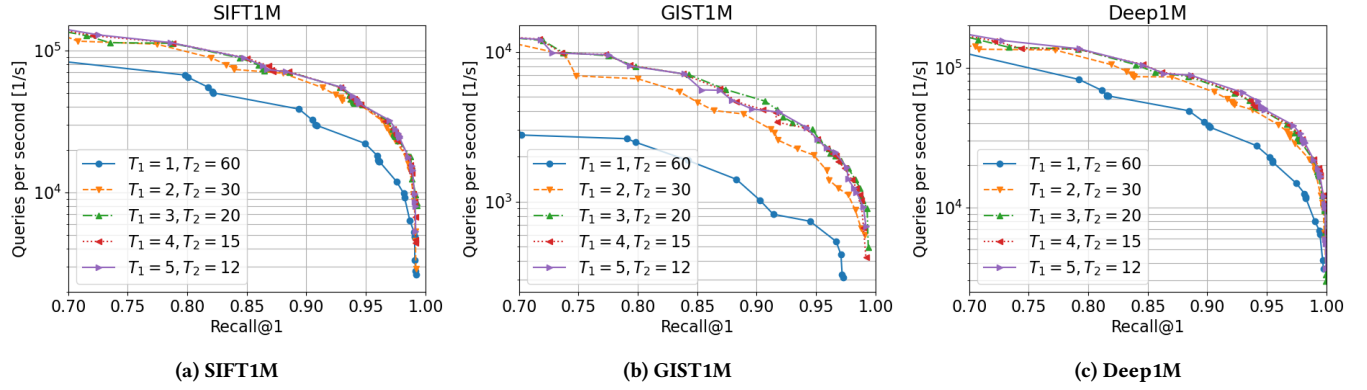
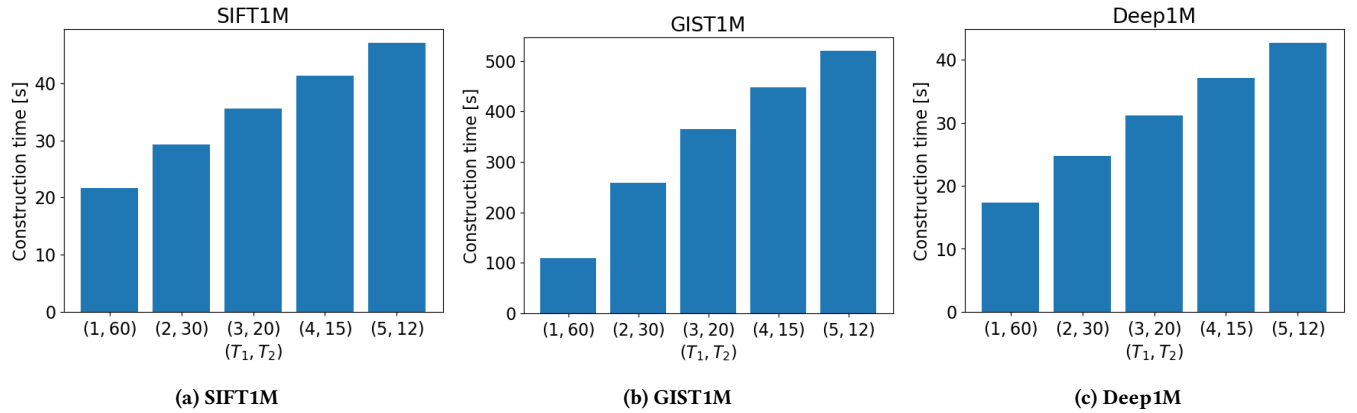
5 EXPERIMENTS

5.1 Settings

This experiment measures the construction time and search performance of each graph-based method. The search accuracy metric is Recall@1 (R@1), where R@1 is the percentage of queries that find the correct nearest neighbor. We also use queries per second (QPS) to measure search speed. In general, ANNS methods have parameters specified during a search, which control the balance between speed and accuracy. We evaluate each method by plotting R@1 and QPS on a plane while varying the search parameters.

We compared the following algorithms:

- NN-Descent [5]: An approximate K-NN graph. We set $K = 64, S = 10, L = 114, R = 100, \text{iter} = 10$.
- NSG [8]: The SOTA of refinement-based approach. NSG uses NN-Descent to construct a K-NN graph. We set $R = 32, L = 64, C = 132$. The parameters of NN-Descent are same as above.
- HNSW [19]: The SOTA of the direct approach. We set $M = 32, \text{efC} = 500$.
- RNN-Descent: The proposed method. We set $S = 20, R = 96, T_1 = 4, T_2 = 15$.

Figure 6: Search performance for various (T_1, T_2) .Figure 7: Construction time for various (T_1, T_2) .

We used the Faiss implementation for comparison. We conducted the experiments on an AWS c6i.4xlarge instance (16 vCPUs, 32 GB memory). We set the number of threads to 16. Table 1 summarizes the properties of the datasets we used.

5.2 Comparison to other methods

Figure 2 and 3 shows each graph-based method’s search performance and construction time. Note that we plot only Pareto-optimal points. The search performance of the proposed method was comparable to the existing SOTA method. On the other hand, the construction time was the shortest among all methods. We emphasize that the construction speed of the proposed method was faster than that of NN-Descent. This result means that existing methods of the refinement-based approach cannot construct the index faster than the proposed method, at least as long as it uses NN-Descent. On the other hand, HNSW belongs to the direct approach and may be faster than the K-NN graph. However, experimental results show that HNSW has the slowest construction time among all the methods.

5.3 Degree distribution

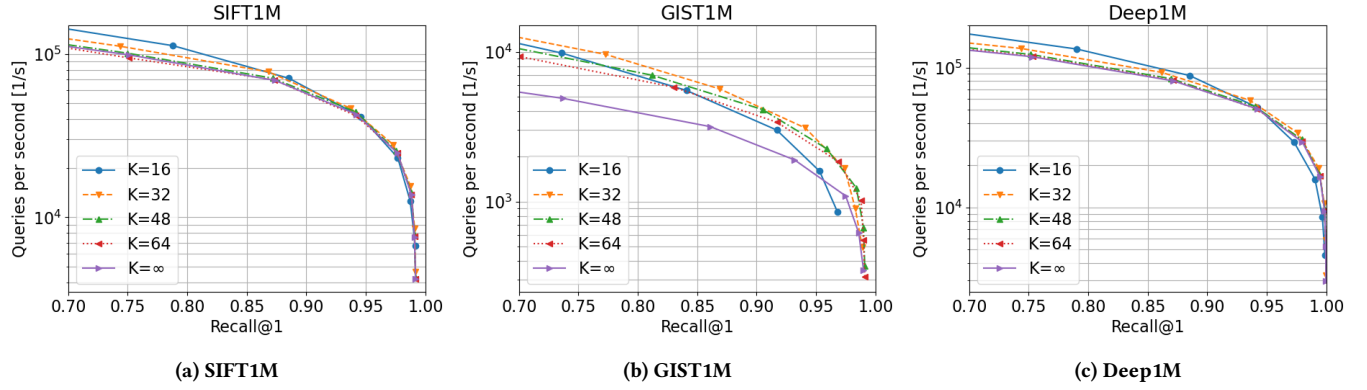
Figures 4 and 5 show the distributions of the in-degree and out-degree of the graphs constructed by each method. The proposed

method limits the in-degree of the graph to be less than or equal to R . However, the average degree was much smaller than R , around 20, and this value was about the same as existing methods. The proposed method has comparable memory efficiency to the existing methods because the memory consumption of the index is proportional to the average degree of the graph.

The degree distributions for SIFT1M and Deep1M were similar to those of NSG. Although the proposed method does not limit the out-degree, the maximum out-degree was around 150. These results demonstrate that the proposed method can automatically adjust the out-degree for relatively simple datasets. On the other hand, for the GIST1M dataset, some vertices have a very large out-degree. These vertices slow the search because the search algorithm checks many neighbors when it visits them. Therefore, the proposed method dynamically limits the out-degree during the search to avoid checking too many neighbors. In addition, the input degree of the proposed method has more concentrated peaks than the input degree of other methods.

5.4 Ablation study

Adding reverse edges. Figures 6 and 7 show the change in search performance and construction speed when T_1 and T_2 are changed.

Figure 8: Search performance for various K .

We keep the total number of iterations $T_1 T_2$ constant. We also set $S = 20$ and $R = 96$ for all experiments. The case $T_1 = 1$ is where the algorithm adds no reverse edges. In this case, the search performance was the lowest of all settings. This result indicates that adding reverse edges is effective in improving search performance. As T_1 increases, the search performance improves while the construction time increases. This result shows that T_1 controls search performance and construction time trade-off.

Limitation on out-degree. Figure 8 shows the change in search performance for different K . We set $K = 16, 32, 48, 64, 96$, and ∞ .

We observe the best K is different whether $R@1$ was greater than approximately 0.95. First, we examine the case when $R@1$ is less than 0.95. The optimal K was 16 for the SIFT1M and Deep1M datasets and 32 for GIST1M. Next, we observe the case when $R@1 > 0.95$. For SIFT1M and Deep1M, the search performance was similar if $K \geq 32$. While for GIST1M, the best K were 48 and 64. These results indicate that K should be small when speed is a priority, and K should be large when accuracy is a priority. In addition, for GIST1M, setting K to ∞ resulted in significant performance degradation due to the significantly large out-degree, as seen in Section 5.3. By setting K appropriately, we can avoid the problem of poor search performance. We emphasize that the user can know the optimal K after index construction. Thus, the proposed method is more robust to changes in the dataset than the conventional methods that require setting the maximum out-degree before construction.

5.5 Experiment for large datasets

We experimented with index construction on a large dataset. We used the SIFT20M dataset, the first 20 million database vectors extracted from the beginning of the SIFT1B [16] dataset (1 billion vectors, 128 dimensions). We conducted the experiments on an AWS c6i.12xlarge (48 vCPUs, 96GB memory) instance. We set the number of threads to 48.

Figure 9 shows the search performance and construction time on the SIFT20M dataset. We compared the proposed method to NSG. The parameters of each method are equal to those described in Section 5.1. First, the proposed method is more than twice as fast as NSG in index construction. On the other hand, the proposed method has comparable search performance to NSG.

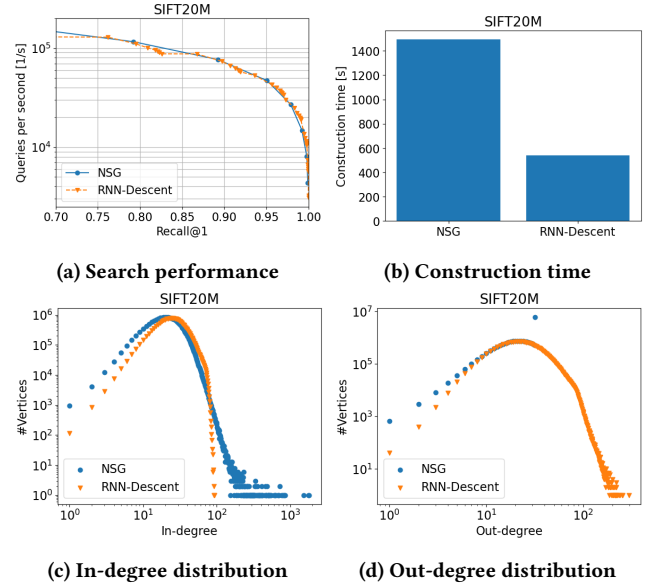


Figure 9: Experiment for SIFT20M dataset.

6 CONCLUSION

This paper proposes RNN-Descent, a new graph-based ANNS index construction algorithm. RNN-Descent combines NN-Descent and RNG Strategy. It simultaneously adds edges based on RNN-Descent and removes edges based on RNG Strategy. Experimental results show that RNN-Descent significantly accelerates index construction while maintaining performance comparable to existing SOTA methods. For example, experiments on the GIST1M dataset show that RNN-Descent constructs the index approximately twice as fast as NSG. Our source code is publicly available on <https://github.com/mti-lab/rnn-descent>.

Acknowledgement. This work was supported by JST AIP Acceleration Research JPMJCR23U2, Japan.

REFERENCES

- [1] Alexandr Andoni, Piotr Indyk, Thijs Laarhoven, Ilya Razenshteyn, and Ludwig Schmidt. 2015. Practical and Optimal LSH for Angular Distance. In *Advances in Neural Information Processing Systems*, C. Cortes, N. Lawrence, D. Lee, M. Sugiyama, and R. Garnett (Eds.), Vol. 28. Curran Associates, Inc. <https://proceedings.neurips.cc/paper/2015/file/2823f4797102ce1a1aec05359cc16dd9-Paper.pdf>
- [2] Martin Aumüller, Erik Bernhardsson, and Alexander Faithfull. 2017. ANN-benchmarks: A benchmarking tool for approximate nearest neighbor algorithms. In *International Conference on Similarity Search and Applications*. Springer, 34–49.
- [3] Artem Babenko and Victor Lempitsky. 2016. Efficient indexing of billion-scale datasets of deep descriptors. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*. 2055–2063.
- [4] Jie Chen, Haw-ren Fang, and Yousef Saad. 2009. Fast Approximate kNN Graph Construction for High Dimensional Data via Recursive Lanczos Bisection. *Journal of Machine Learning Research* 10, 9 (2009).
- [5] Wei Dong, Charikar Moses, and Kai Li. 2011. Efficient k-nearest neighbor graph construction for generic similarity measures. In *Proceedings of the 20th international conference on World wide web*. 577–586.
- [6] Cong Fu and Deng Cai. 2016. Efanna: An extremely fast approximate nearest neighbor search algorithm based on knn graph. *arXiv preprint arXiv:1609.07228* (2016).
- [7] Cong Fu, Changxu Wang, and Deng Cai. 2022. High Dimensional Similarity Search With Satellite System Graph: Efficiency, Scalability, and Unindexed Query Compatibility. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 44, 8 (2022), 4139–4150. <https://doi.org/10.1109/TPAMI.2021.3067706>
- [8] Cong Fu, Chao Xiang, Changxu Wang, and Deng Cai. 2019. Fast Approximate Nearest Neighbor Search With The Navigating Spreading-out Graphs. *PVLDB* 12, 5 (2019), 461 – 474. <https://doi.org/10.14778/3303753.3303754>
- [9] Tiezheng Ge, Kaiming He, Qifa Ke, and Jian Sun. 2014. Optimized Product Quantization. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 36, 4 (2014), 744–755. <https://doi.org/10.1109/TPAMI.2013.240>
- [10] Aristides Gionis, Piotr Indyk, Rajeev Motwani, et al. 1999. Similarity search in high dimensions via hashing. In *Vldb*, Vol. 99. 518–529.
- [11] Ben Harwood and Tom Drummond. 2016. Fanng: Fast approximate nearest neighbour graphs. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*. 5713–5722.
- [12] Masajiro Iwasaki. 2016. Pruned bi-directed k-nearest neighbor graph for proximity search. In *International Conference on Similarity Search and Applications*. Springer, 20–33.
- [13] Masajiro Iwasaki and Daisuke Miyazaki. 2018. Optimization of indexing based on k-nearest neighbor graph for proximity search in high-dimensional data. *arXiv preprint arXiv:1810.07355* (2018).
- [14] Suhas Jayaram Subramanya, Fnu Devvrit, Harsha Vardhan Simhadri, Ravishankar Krishnaswamy, and Rohan Kadekodi. 2019. Diskann: Fast accurate billion-point nearest neighbor search on a single node. *Advances in Neural Information Processing Systems* 32 (2019).
- [15] Herve Jegou, Matthijs Douze, and Cordelia Schmid. 2010. Product quantization for nearest neighbor search. *IEEE transactions on pattern analysis and machine intelligence* 33, 1 (2010), 117–128.
- [16] Jeff Johnson, Matthijs Douze, and Hervé Jégou. 2019. Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data* 7, 3 (2019), 535–547.
- [17] Wen Li, Ying Zhang, Yifang Sun, Wei Wang, Mingjie Li, Wenjie Zhang, and Xuemin Lin. 2019. Approximate nearest neighbor search on high dimensional data—experiments, analyses, and improvement. *IEEE Transactions on Knowledge and Data Engineering* 32, 8 (2019), 1475–1488.
- [18] Yury Malkov, Alexander Ponomarenko, Andrey Logvinov, and Vladimir Krylov. 2014. Approximate nearest neighbor algorithm based on navigable small world graphs. *Information Systems* 45 (2014), 61–68.
- [19] Yu A Malkov and Dmitry A Yashunin. 2018. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE transactions on pattern analysis and machine intelligence* 42, 4 (2018), 824–836.
- [20] Yusuke Matsui, Martin Aumüller, and Han Xiao. 2023. CVPR2023 Tutorial on Neural Search in Action. https://matsui528.github.io/cvpr2023_tutorial_neural_search/.
- [21] Marius Muja and David G Lowe. 2009. Fast approximate nearest neighbors with automatic algorithm configuration. *VISAPP (1)* 2, 331–340 (2009), 2.
- [22] Aditi Singh, Suhas Jayaram Subramanya, Ravishankar Krishnaswamy, and Harsha Vardhan Simhadri. 2021. FreshDiskANN: A Fast and Accurate Graph-Based ANN Index for Streaming Similarity Search. *arXiv preprint arXiv:2105.09613* (2021).
- [23] Jing Wang, Jingdong Wang, Gang Zeng, Zhuowen Tu, Rui Gan, and Shipeng Li. 2012. Scalable k-NN graph construction for visual descriptors. In *CVPR 2012*. 1106–1113.
- [24] Jingdong Wang, Naiyan Wang, You Jia, Jian Li, Gang Zeng, Hongbin Zha, and Xian-Sheng Hua. 2014. Trinary-Projection Trees for Approximate Nearest Neighbor Search. *IEEE Trans. Pattern Anal. Mach. Intell.* 36, 2 (2014), 388–403.
- [25] Mengzhao Wang, Xiaoliang Xu, Qiang Yue, and Yuxiang Wang. 2021. A Comprehensive Survey and Experimental Comparison of Graph-Based Approximate Nearest Neighbor Search. *Proc. VLDB Endow.* 14, 11 (oct 2021), 1964–1978. <https://doi.org/10.14778/3476249.3476255>
- [26] Wan-Lei Zhao, Hui Wang, and Chong-Wah Ngo. 2021. Approximate k-NN graph construction: a generic online approach. *IEEE Transactions on Multimedia* 24 (2021), 1909–1921.